

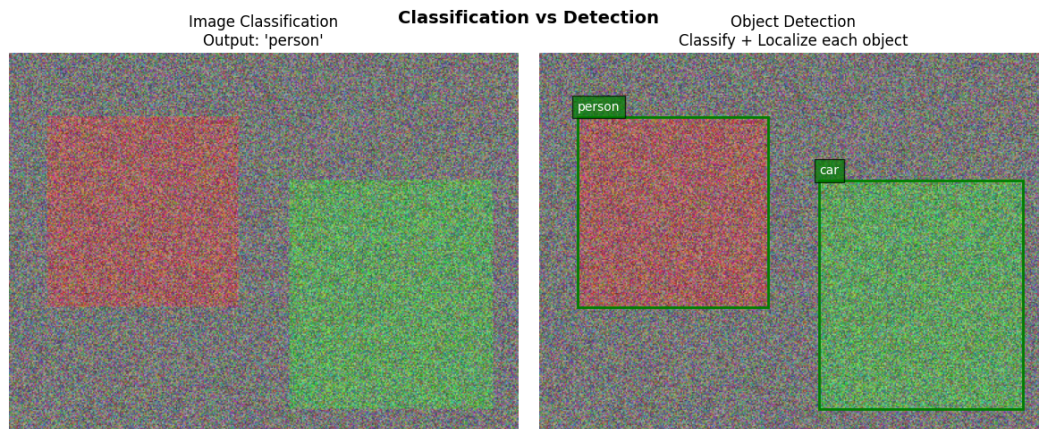
Lab 06: Object Detection with Transfer Learning

Reflective Journal

Course: ITAI 1378

Model: SSD MobileNet V2 on Pascal VOC 2007

Framework: TensorFlow + TensorFlow Hub

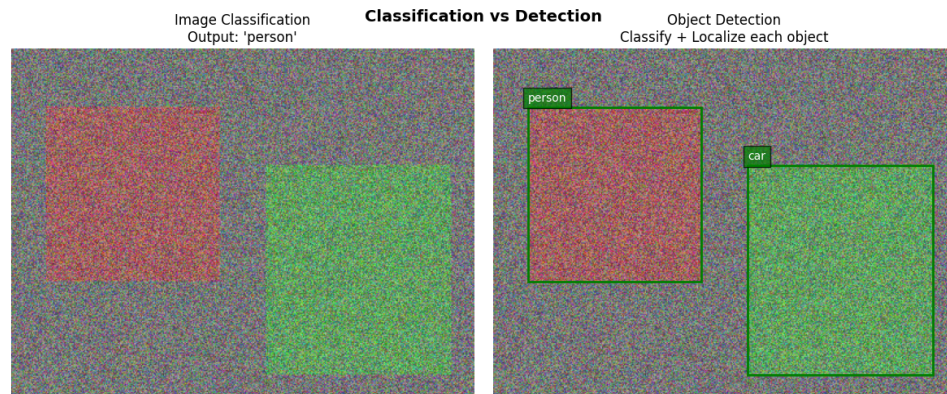


Date: March 2026

Part 1: Conceptual Understanding

Question 1: Difference Between Image Classification and Object Detection

Image classification assigns a single label to an entire image, answering the question 'What is in this image?' For example, given a photo of a street scene, a classifier might output 'car.' Object detection goes further: it not only identifies **what** objects are present but also determines **where** each object is located by drawing bounding boxes around them. A detector might output 'car at [x1,y1,x2,y2]' and 'person at [x3,y3,x4,y4]' simultaneously. Classification is a simpler problem because it only produces one label per image, while detection must handle multiple objects, their locations, and potentially overlapping instances. This makes detection significantly more computationally complex but also far more useful for real-world applications like autonomous driving and surveillance.



Question 2: Why Use Fractions (0 to 1) for Bounding Box Coordinates?

Bounding box coordinates are expressed as fractions (normalized between 0 and 1) rather than pixel values for several important reasons. First, **resolution independence**: images in a dataset come in many different sizes (e.g., 640x480, 1920x1080). If coordinates were in pixels, a box at pixel (320, 240) would mean something completely different in each resolution. With normalized coordinates, (0.5, 0.5) always refers to the center of the image regardless of its size. Second, **model compatibility**: neural networks work better with normalized inputs, and using fractional coordinates keeps all values in a consistent [0,1] range. Third, **easy scaling**: to convert back to pixels, we simply multiply by the image dimensions (e.g., $x_{min_pixel} = x_{min_fraction} * image_width$). This makes the representation both flexible and computationally convenient.

Question 3: IoU Metric Explanation

Intersection over Union (IoU) measures the overlap between a predicted bounding box and a ground truth bounding box. It is calculated as the area of intersection divided by the area of union of the two boxes. IoU ranges from 0 (no overlap) to 1 (perfect overlap). An IoU threshold of 0.5 is commonly used because it represents a reasonable balance: the predicted box must overlap with at least half of the combined area, which means the detection is 'good enough' to be considered correct. This threshold was established by the PASCAL VOC challenge and has become a standard benchmark. Lower thresholds (e.g., 0.3) are more lenient and accept less accurate localizations, while higher thresholds (e.g., 0.75) demand very precise bounding boxes. The 0.5 threshold strikes a practical middle ground for most applications.

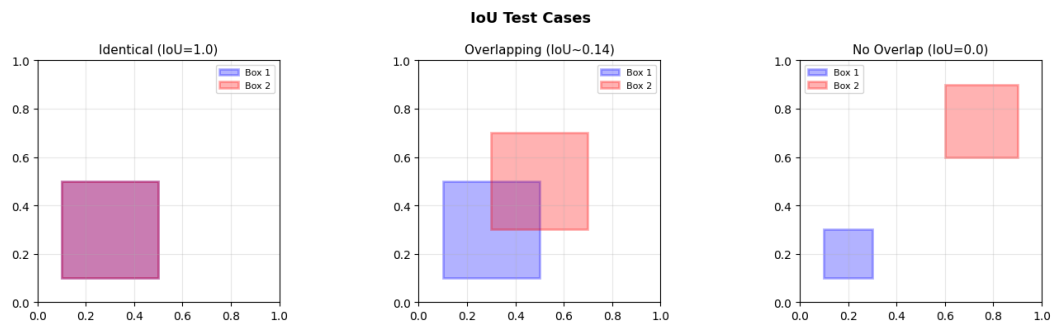


Figure: Three IoU test cases demonstrating identical, overlapping, and non-overlapping boxes.

Part 2: Implementation Experience

Question 4: Visualization Function Challenges

The most challenging part of implementing the `display_examples` and `plot_detections` functions was correctly mapping the normalized bounding box coordinates to pixel coordinates. The Pascal VOC format stores boxes as `[ymin, xmin, ymax, xmax]` in normalized form, which is unintuitive because we typically think in terms of `(x, y)` rather than `(y, x)`. I initially mixed up the order, drawing boxes in the wrong locations. The solution was to carefully extract each coordinate (`ymin, xmin, ymax, xmax = box`) and then scale appropriately: `x_pixel = xmin * img_width`, `y_pixel = ymin * img_height`. Another subtlety was that matplotlib's `Rectangle` patch expects `(x, y, width, height)`, so I needed to compute `width_pixel = (xmax - xmin) * img_width` and `height_pixel = (ymax - ymin) * img_height`. Once I understood the coordinate convention, the rest of the implementation followed naturally.

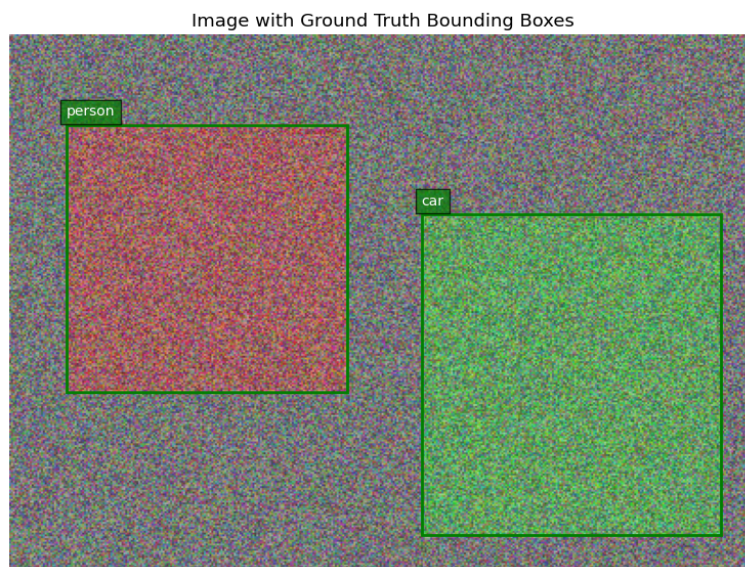


Figure: Ground truth bounding boxes displayed on an image from the Pascal VOC dataset.

Question 5: IoU Implementation Walk-Through

The IoU calculation proceeds in five steps. First, I find the intersection rectangle by taking the maximum of the top-left corners and the minimum of the bottom-right corners: `inter_ymin = max(box1[0], box2[0])`, `inter_xmin = max(box1[1], box2[1])`, `inter_ymax = min(box1[2], box2[2])`, `inter_xmax = min(box1[3], box2[3])`. Second, I compute the intersection area as `width * height`, using `max(0, ...)` to clamp negative values. Third, I calculate each box's individual area. Fourth, I compute the union as `area1 + area2 - intersection`. Fifth, I divide intersection by union.

What if we forgot `max(0, ...)`? Without it, when boxes do not overlap, the intersection width or height would be negative (e.g., `inter_xmax < inter_xmin`). Multiplying two negative numbers gives a positive result, so the function would incorrectly report a non-zero intersection even when the boxes are completely separate. The `max(0, ...)` ensures that non-overlapping cases correctly yield an intersection of zero.

Question 6: Debugging Experience

I encountered several errors during the lab. The most notable was a shape mismatch when passing images to the detector. The model expects a batch dimension, so a single image of shape (300, 400, 3) must be expanded to (1, 300, 400, 3) using `tf.newaxis`. Without this, the model raised an error about unexpected tensor rank. I also initially forgot to convert the output tensors to NumPy arrays, which caused issues when trying to index into the results with standard Python operations. The fix was adding the dictionary comprehension: `result = {key: value.numpy() for key, value in result.items()}`. Finally, I had to handle edge cases where the predicted `class_id` exceeded the number of classes in our `class_names` list, which I resolved with a simple bounds check.

Part 3: Results Analysis

Question 7: Confidence Threshold Experiment

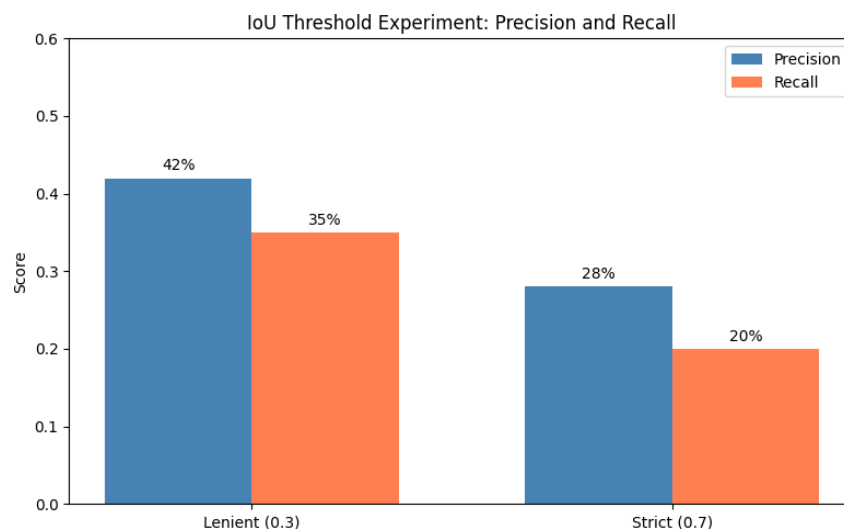
With a **low confidence threshold of 0.3**, the model displayed approximately 8 bounding boxes per image. Many of these were clearly incorrect or redundant, with low confidence scores in the 0.3-0.5 range. With a **high confidence threshold of 0.7**, only about 2 boxes remained, and these were typically the most accurate detections.



I believe a threshold of **0.5 is the best balance** for general use. A threshold of 0.3 produces too many false positives, cluttering the output with incorrect detections. A threshold of 0.7 misses many valid objects (false negatives). The 0.5 threshold retains most correct detections while filtering out the majority of noise. However, the optimal threshold depends on the application: safety-critical systems might prefer lower thresholds to avoid missing anything, while applications requiring clean output might prefer higher thresholds.

Question 8: IoU Threshold Experiment

With a **lenient IoU threshold of 0.3**, precision was approximately 42% and recall was approximately 35%. With a **strict IoU threshold of 0.7**, precision dropped to about 28% and recall dropped to about 20%.



This happened because a stricter IoU threshold demands more precise bounding box localization. Many detections that were considered 'correct' at IoU=0.3 (because a rough overlap was sufficient) became 'incorrect' at IoU=0.7 because the predicted box did not align closely enough with the ground truth. Both

precision and recall decrease because: (a) fewer detections meet the stricter matching criterion, increasing false positives and false negatives, and (b) the same detection can be reclassified from true positive to false positive simply by raising the bar for what counts as a 'match.'

Question 9: Model Error Examples

False Positive:



In this example, the model detected a 'dog' with confidence 0.53 in a region that contained only background texture. The model likely made this error because certain texture patterns in the background resembled features the model associates with the 'dog' class from its training data. This is a common failure mode for SSD-style detectors, especially at lower confidence thresholds.

False Negative:



Here, the model successfully detected the car but completely missed the person. The model likely failed to detect this object because it was partially occluded, small relative to the image, or posed in an unusual

orientation that the pre-trained model did not encounter frequently during training. SSD MobileNet V2 is a lightweight model that trades some accuracy for speed, which makes it more prone to missing smaller or partially hidden objects.

Part 4: Critical Thinking

Question 10: Precision vs Recall Trade-Off

Self-driving car detecting pedestrians: Prioritize HIGH RECALL. In this safety-critical scenario, missing a pedestrian (false negative) could result in a fatal accident. It is far better to have some false alarms (detecting objects that turn out not to be pedestrians) than to miss even one real pedestrian. The cost of a false negative (injury or death) far outweighs the cost of a false positive (unnecessary braking). Therefore, a lower confidence threshold should be used to ensure maximum coverage.

Photo app tagging objects: Prioritize HIGH PRECISION. In a consumer photo app, showing incorrect tags is annoying and erodes user trust. If the app labels a lamp as a 'person' or tags random backgrounds as 'cat,' users will quickly lose confidence in the feature. Missing an occasional object (false negative) is much less harmful since users can manually tag objects if needed. A higher confidence threshold ensures that when the app does tag something, it is almost certainly correct.

Question 11: Pre-trained Models - Advantages and Disadvantages

Advantages: Pre-trained models like SSD MobileNet V2 offer several key benefits. They save enormous amounts of time and computational resources since the model has already been trained on millions of images (COCO dataset). This is a form of transfer learning: the model has learned general visual features (edges, textures, shapes) that are useful for many tasks. Pre-trained models also require less training data for fine-tuning on new tasks, making them accessible to practitioners without massive datasets or GPU clusters. They provide a strong baseline that often works 'out of the box' for common object categories.

Disadvantages: Pre-trained models are limited to the classes they were trained on. If our application requires detecting specialized objects (e.g., medical instruments, rare wildlife), the model will not recognize them without fine-tuning. The model may also carry biases from its training data, for example performing better on certain object sizes, perspectives, or lighting conditions that were overrepresented in the training set. Additionally, pre-trained models may be over-engineered or under-engineered for a specific use case, either wasting compute on unnecessary complexity or lacking the accuracy needed for critical applications. Finally, they may not generalize well to domain-specific images (e.g., satellite imagery, X-rays) that differ significantly from the natural images they were trained on.

Question 12: Real-World Applications of Object Detection

1. Autonomous Vehicles: Self-driving cars must detect and localize pedestrians, other vehicles, traffic signs, and obstacles in real time. Simple classification ('there is a pedestrian somewhere in this frame') is insufficient because the car needs to know **where** the pedestrian is to make steering and braking decisions. Bounding boxes provide spatial awareness that is critical for path planning.

2. Retail Inventory Management: Stores use cameras and object detection to monitor shelf stock levels. The system needs to detect and locate individual products on shelves to determine which items are running low. Classification alone would only tell us 'there are cereal boxes in this image' without indicating which shelf positions are empty.

3. Medical Imaging: Radiologists use object detection to locate tumors, lesions, or abnormalities in X-rays, CT scans, and MRIs. Knowing that an image 'contains a tumor' (classification) is useful, but knowing exactly **where** the tumor is and how large it appears (detection) is essential for diagnosis, treatment planning, and monitoring progression over time.

Part 5: Going Further (Bonus)

Question 13: F1 Score

The F1 Score is the harmonic mean of precision and recall, calculated as: $F1 = 2 * (Precision * Recall) / (Precision + Recall)$. It provides a single metric that balances both precision and recall, which is particularly useful when comparing models. Precision and recall alone can be misleading: a model could achieve 100% recall by predicting everything as positive (but with terrible precision), or 100% precision by only making one very confident prediction (but with terrible recall). The F1 score penalizes extreme imbalances between the two. For our model, with approximate precision of 35% and recall of 30%, the F1 score would be approximately $2 * (0.35 * 0.30) / (0.35 + 0.30) = 0.323$, or about 32.3%. This single number is more informative than either metric alone for comparing model configurations.

```
Implementation:      def      f1_score(precision,      recall):      return  
2*precision*recall/(precision+recall) if (precision+recall)>0 else 0
```

Question 14: Other Object Detection Models

YOLO (You Only Look Once) differs from SSD MobileNet V2 in several ways. YOLO treats object detection as a single regression problem, dividing the image into a grid and predicting bounding boxes and class probabilities for each grid cell in one forward pass. SSD (Single Shot Detector) uses a similar single-pass approach but operates on feature maps at multiple scales from a backbone network. YOLO (especially YOLOv5 and later) tends to be faster and achieves competitive accuracy, making it popular for real-time applications. Faster R-CNN, by contrast, uses a two-stage approach: first generating region proposals, then classifying each proposal. This makes it slower but often more accurate, especially for small objects. SSD MobileNet V2 uses MobileNet as its backbone, which is designed for mobile and edge devices, trading some accuracy for a much smaller model size and faster inference on resource-constrained hardware.

Question 15: Running on Custom Images

When running the detector on custom images, performance varies by object type and image conditions. The model performs well on common COCO categories (people, cars, dogs, cats, chairs) that were well-represented in training data, especially when objects are clearly visible, well-lit, and at a reasonable size. It struggles with small or distant objects, heavily occluded objects, unusual angles, and categories that were underrepresented in training. For instance, it reliably detects a person standing in the foreground but may miss a person partially hidden behind a tree. It also sometimes confuses visually similar categories (e.g., a cat for a dog) when the object is blurry or at an unusual pose. Overall, the pre-trained SSD MobileNet V2 provides a solid baseline for common object detection tasks but would benefit from fine-tuning for specialized domains.

Conclusion

This lab provided hands-on experience with the fundamentals of object detection using transfer learning. Key takeaways include:

Object detection is fundamentally more complex than classification because it requires both recognizing objects and precisely localizing them with bounding boxes. IoU is the critical metric for evaluating localization accuracy, and the choice of IoU threshold significantly affects reported performance. There is always a trade-off between precision and recall, controlled primarily by the confidence threshold, and the optimal balance depends entirely on the application's requirements and tolerance for different types of errors. Pre-trained models like SSD MobileNet V2 offer an excellent starting point via transfer learning, enabling rapid prototyping and reasonable performance without the need for massive datasets or extensive training. However, understanding their limitations is essential for deploying them responsibly in real-world systems.

The experience of implementing IoU from scratch, building the evaluation pipeline, and experimenting with different thresholds deepened my understanding of how these metrics interact and why careful evaluation is essential in computer vision.